

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №11

по дисциплине: Основы программирования

тема: **«Рекурсивные функции»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2022 г.

Цель работы: получение навыков написания функций для решения задач со структурами.

Содержание отчета:

Тема лабораторной работы.

Цель лабораторной работы.

Решения задач:

- Текст задания.
- Исходный код.
- Решения задач с двумя звёздочками являются необязательными

Структура Point

a)

```
Point p1 = {2, 3};
```

b)

```
//функция для ввода структуры p типа Point
void inputPoint(Point *p) {
    scanf("%lf %lf", &p->x, &p->y);
}
```

c)

```
//функция для вывода структуры p типа Point
void outputPoint(Point p) {
    printf("%.3lf %.3lf", p.x, p.y);
}
```

d)

```
Point p1 = {2, 4};
Point p2 = p1;
```

e)

```
#define N 3

//функция для ввода массива структур p типа Point размера n
void inputPoints(Point *p, int n) {
    for (int i = 0; i < n; i++) {
        printf("print x and y : \n");
        inputPoint(&p[i]);
    }
}

//функция для вывода массива структур p типа Point размера n
void outputPoints(Point *p, int n) {
    for (int i = 0; i < n; i++) {
        outputPoint(p[i]);
        printf("\n");
    }
}

int main() {
    //a)
    Point p1[N];

    inputPoints(p1, N);

    outputPoints(p1, N);

    return 0;
}
```

f)

```
//функция которая принимает две структуры p1 и p2 типа Point и возвращает
точку посередине
Point getMiddlePoint(Point p1, Point p2) {
    return (Point) {
        (p1.x + p2.x) / 2,
        (p1.y + p2.y) / 2
    };
}
```

g)

```
//функция которая принимает две структуры p1 и p2 типа Point и возвращает
'истина', если точки совпадают
int arePointsEqual(Point p1, Point p2) {
    return p1.x == p2.x && p1.y == p2.y;
}
```

h)

```
//функция которая возвращает 'истина', если точка p3 лежит посередине между
точками p1 и p2
int isPointBetween(Point p1, Point p2, Point p3) {
    Point check = getMiddlePoint(p1, p2);

    return (arePointsEqual(check, p3));
}
```

i)

```
//функция которая обменивает значения x и y в структуре p типа Point
void swapCoordinates(Point *p) {
    double t = p->x;
    p->x = p->y;
    p->y = t;
}
```

j)

```
//функция которая обменивает значения двух структур p1 и p2 типа Point
void swapPoints(Point *p1, Point *p2) {
    swapForPoint(&p1->x, &p2->x, sizeof(p1->x));
    swapForPoint(&p1->y, &p2->y, sizeof(p1->y));
}
```

k)

```
Point *p1 = (Point*)malloc(sizeof(Point) * N);
free(p1);
```

l)

```
//функция которая находит расстояние между точками p1 и p2 типа Point
double getDistance(Point p1, Point p2) {
    double x = p2.x - p1.x;
    double y = p2.y - p1.y;

    return sqrt(x * x + y * y);
}
```

Структура Circle

a)

```
Circle c = {(Point){1, 2}, 5};
```

b)

```
Circle c1[] = {(Point){1, 2}, 3, (Point){1, 2}, 3};
```

c)

```
//функция для ввода структуры а типа Circle
void inputCircle(Circle *a) {
    inputPoint(&a->center);
    scanf("%lf", &a->r);
}
```

d)

```
//функция для ввода массива структур а типа Circle размера n
void inputCircles(Circle *a, size_t n) {
    for (int i = 0; i < n; i++) {
        printf("print x, y and c : \n");
        inputCircle(&a[i]);
    }
}
```

e)

```
//функция для вывода структуры а типа Circle
void outputCircle(Circle a) {
    outputPoint(a.center);
    printf(" %.3lf", a.r);
}
```

f)

```
//функция для вывода массива структур а типа Circle размера n
void outputCircles(Circle *a, size_t n) {
    for (int i = 0; i < n; i++) {
        outputCircle(a[i]);
        printf("\n");
    }
}
```

g)

```
//функция которая возвращает 'истина', если окружность c1 касается внешним
образом окружность c2
int hasOneOuterIntersection(Circle c1, Circle c2) {
    int len = getDistance(c1.center, c2.center);

    return len == c1.r + c2.r ? 1 : 0;
}
```